SCOPE: Streaming Covariance-Orthogonal Patching for Efficient LLM Safety Governance

Anonymous Author(s)

Abstract

Deployed large language models require continuous post-hoc safety updates to counter evolving threats. However, existing interventions often entangle refusal behaviors with capability-critical representations, causing cumulative utility drift under sequential edits. We propose SCOPE, a streaming post-hoc editing framework that enforces representation-level safety governance via capability invariance. SCOPE estimates layer-wise capability subspaces from online activation covariance and constrains all safety updates to their orthogonal complements, separating capability preservation from safety-specific edits. Within this constrained null space, SCOPE extracts low-rank refusal directions from contrastive harmful and benign activations and steers refusals without interfering with protected capability regions. To support continual deployment, SCOPE maintains activation covariances online, enabling expansion of the protected capability subspace under open-ended threat evolution without expensive recomputation. Experiments show that, across model backbones, SCOPE reduces SORRYBench ASR by up to **65%**, while preserving MMLU and LiveCodeBench within natural run-to-run variance under repeated post-deployment edits, enabling low-latency and auditable safety maintenance. Code is available at https://anonymous.4open.science/r/SCOPE-657A.

CCS Concepts

• **Computing methodologies** → **Language models**; *Machine learning algorithms*; *Representation learning*.

Keywords

Safety Governance, Alignment Tax, Streaming Model Patching, Large Language Models

ACM Reference Format:

Anonymous Author(s). 2026. SCOPE: Streaming Covariance-Orthogonal Patching for Efficient LLM Safety Governance. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large language models (LLMs) are increasingly deployed as persistent, general-purpose knowledge infrastructures [54–57, 60], serving heterogeneous downstream applications over long lifecycles characterized by continuous updates [1, 4]. In such dynamic environments, post-deployment safety failures incur not only societal risks but also substantial operational instabilities, including service disruptions and cross-domain performance regressions, making sustained model governance a critical challenge [5, 51, 61].

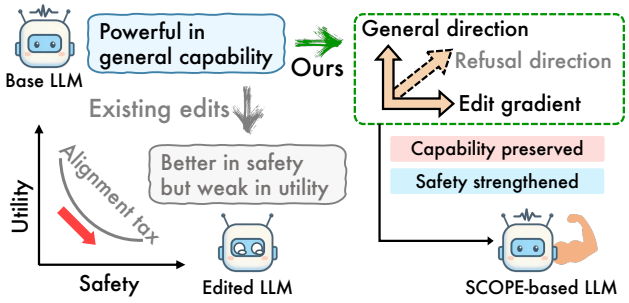

Figure 1: SCOPE enforces post-deployment safety updates by projecting refusal-oriented weight edits onto the orthogonal complement of empirically estimated capability subspaces, thereby explicitly separating preserved capabilities from safety-specific modifications.

Unlike static pre-training or periodic full retraining, deployed systems necessitate repeated, incremental safety patches to counter evolving jailbreak attacks and shifting policy requirements over time [9, 34, 49]. Crucially, these interventions must satisfy a strict *deployment invariant*: safety behaviors must be strengthened without compromising the representational structures that support general capabilities [13, 35, 37]. However, many existing safety update strategies fundamentally violate this requirement by entangling safety objectives with general-purpose representations, producing models that are more robust yet systematically weaker in overall utility [11, 31]. Figure 1 illustrates this structural tension by contrasting conventional safety edits that trade capability for robustness with a geometric perspective that separates safety-specific modifications from preserved capabilities. This deployment-invariant constraint exposes a fundamental limitation of existing alignment methods under repeated post-deployment updates [15, 28].

While standard approaches such as instruction tuning and reinforcement learning from human feedback (RLHF) improve refusal robustness [3, 38], they inherently impose an alignment tax [31], trading safety gains for degraded utility across unrelated downstream tasks. In deployed settings, repeated fine-tuning under evolving safety objectives accumulates uncontrolled interference, leading to gradual capability drift that is difficult to localize, audit, or reverse [1, 27, 39, 44]. Consequently, without an explicit preservation mechanism, merely increasing update frequency or data coverage provides limited mitigation against such degradation [15, 39].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

However, this challenge is further exacerbated by the limited control offered by existing mitigation strategies [9, 34], as training-based methods complicate post-hoc auditing by globally shifting model parameters [1, 44]. Although post-hoc model editing enables rapid interventions via localized parameter updates [35–37], most existing editors are tailored for discrete factual rewrites rather than sustained, behavior-level refusal control required for continual safety governance [8, 47]. Furthermore, recent null-space constrained methods [13] primarily enforce *parameter locality* rather than *functional invariance*, failing to guarantee the stability of deployed capabilities under sequential patching. As a result, cumulative unintended interference remains a largely unmanaged risk in long-term deployment scenarios [44].

To enable effective post-deployment safety updates, it is essential to explicitly define which capabilities must be preserved. To address this gap, we propose **SCOPE** (**S**treaming **C**ovariance-**O**rthogonal **P**atching **E**diting), a framework that reframes post-deployment safety alignment as a system identification problem under explicit invariance constraints. Specifically, SCOPE models a deployed LLM as a system with a statistically stable capability profile, estimated via layer-wise second-order activation statistics. Safety updates are strictly projected onto the orthogonal complement of this capability subspace, enforcing a geometric separation between preserved general knowledge and edited safety behaviors without costly re-training. Building on this invariance constraint, SCOPE treats refusal as a structured, steerable behavioral objective. Recognizing that effective refusal in deployed systems ranges from explicit rejection to constructive redirection (Figure 2), SCOPE extracts low-rank refusal directions from contrastive activation pairs. To enable controllable strengthening of these behaviors, SCOPE then steers model behavior strictly within the safe null-space. To support continuous deployment, SCOPE employs online covariance updates that allow the capability subspace to adapt incrementally to distribution shifts, enabling low-latency safety patching without full-dataset recomputation. Our main contributions are:

- **Explicit Capability Preservation**: We formalize capability preservation as an invariance constraint via layer-wise activation covariance estimation, which decouples safety editing from utility loss and enables direct optimization.
- **Orthogonal Refusal Patching**: We develop a null-space projected weight editing technique that strengthens mechanistic refusal behaviors while suppressing interference with general capabilities, ensuring robust safety enforcement without compromising the model's original performance.
- **Continual Safety Maintenance**: We propose an efficient online covariance update mechanism that supports scalable, low-latency safety patching adaptable to domain drift, enabling sustainable and efficient model safety updates in dynamic real-world post-development environments.

2 Related Works

2.1 Post-hoc editing with capability invariance

Post-hoc model editing enables targeted parameter updates without full model retraining, offering a practical pathway for deployment-time intervention [8, 37, 47]. Representative methods include learned meta-editors that predict localized weight updates [37] and direct editing methods that rewrite internal factual associations through rank-constrained parameter modifications [35, 36]. While these methods demonstrate strong locality under one-shot edits, they are fundamentally designed for factual correction and lack the representational scaffolding needed for behavior-level safety updates under repeated patching regimes [16, 49, 53].

Figure 2: Two forms of refusal behavior observed in deployed systems: explicit refusal directly rejects harmful requests, while constructive refusal provides explanation and redirects user intent under safety constraints.

To enhance robustness across sequential edits, recent work has introduced null-space constrained editing, which enforces update locality to mitigate cross-edit interference [7, 13, 50]. Although effective in reducing cross-edit interference, these approaches remain update-centric and lack an explicit invariance target that defines which deployed capabilities must be preserved [16, 30]. As a result, the preservation of general or domain-specific behaviors remains implicit and difficult to guarantee under continual safety updates, particularly in long-lived deployment settings.

Addressing this gap, SCOPE operationalizes capability preservation directly at the representation level: it first models deployed capabilities through data-driven activation statistics, then explicitly constrains safety updates to the orthogonal complements of the identified capability subspaces, thereby ensuring that essential functionalities remain invariant even under repeated editing.

2.2 Safety alignment in deployment

Most safety alignment pipelines enhance refusal behavior through training-based approaches, including instruction tuning and reinforcement learning from human feedback, typically guided by iterative red-teaming [3, 38, 40]. While these methods effectively improve refusal robustness, they inherently entangle safety objectives with the model's general-purpose representations, frequently imposing a significant alignment tax on general capability in deployed systems [31], as repeated fine-tuning under evolving safety objectives accumulates uncontrolled interference, leading to gradual capability drift that is difficult to localize, audit, or reverse [21, 27, 39]. Alternative mitigation strategies, such as prompt filtering and input-based detection, operate externally and do not strengthen the model's intrinsic refusal mechanisms [23, 59].

In contrast, SCOPE performs safety adaptation directly at the parameter level via post-hoc edits that are explicitly constrained by capability invariance. This enables repeated post-deployment safety updates that strengthen refusal behavior while avoiding retraining and cumulative utility degradation.

2.3 Projection-based refusal control

Projection-based optimization has been widely studied as a mechanism for mitigating interference in multi-task and continual learning [26, 32, 39, 45]. Representative examples include projecting conflicting gradients in multi-task learning [58] and preserving previously learned behaviors via orthogonal gradient descent in continual learning [14, 43]. While such methods demonstrate the utility of projection in reducing interference, they do not explicitly specify which internal representations should be preserved for deployment-time safety patching [27, 39, 42]. Separately, recent mechanistic analyses indicate that refusal behavior can be steered via low-dimensional structure in the model's representation space when relevant directions are precisely identified [2]. However, mechanistic steering alone does not guarantee the preservation of broader model capabilities under repeated safety updates [6, 29, 48].

SCOPE bridges these lines of work by integrating data-driven capability subspaces with projection-based optimization, enforcing null-space constrained safety edits, and supporting continual expansion of the protected subspace as new safety domains emerge. This design enables behavior-specific refusal control while maintaining stable utility under repeated post-deployment updates.

3 Methodology

SCOPE implements a **Discover → Govern → Expand** workflow for post-hoc safety patching of autoregressive LLMs as follows:

- **Discover:** A capability subspace is identified from activation statistics, establishing an orthogonal null space for safe editing under strict geometric constraints.
- **Govern:** Safety behaviors are steered by aligning interface activations with a learned refusal direction and applying null-space projected updates, strictly confining edits to $\mathcal{S}_{\text{null}}$ while preserving $\mathcal{S}_{\text{gen}}$.
- **Expand:** The capability boundaries are dynamically extended via online covariance maintenance, enabling incremental null-space expansion to accommodate distribution shifts without recomputation.

3.1 Discover: general capability subspace via activation statistics

To ensure that the model's core functional capabilities remain invariant during safety-oriented editing, we first identify the underlying representation subspace that encodes these capabilities. This is achieved by collecting input activation vectors $\mathbf{h}_i^{(\ell)} \in \mathbb{R}^{d_\ell}$ at each layer ℓ, where i corresponds to one of the n_ℓ tokens sampled from a general-purpose corpus $\mathcal{D}_{\text{cap}}$. From these activations, we construct a regularized empirical covariance matrix to capture their second-order statistical structure:

$$\mathbf{C}^{(\ell)} = \lambda \mathbf{I}_{d_\ell} + \sum_{i=1}^{n_\ell} \mathbf{h}_i^{(\ell)} \mathbf{h}_i^{(\ell)\top}, \tag{1}$$

where $\lambda > 0$ is a ridge regularization parameter that ensures numerical stability, and $\mathbf{I}_{d_\ell}$ denotes the $d_\ell \times d_\ell$ identity matrix. Based on this procedure, an eigendecomposition of $\mathbf{C}^{(\ell)}$ produces a set of eigenvectors, ordered by their corresponding eigenvalues.

To balance the preservation of capabilities against the flexibility required for editing, we define a truncation rank r_ℓ. Specifically, r_ℓ is determined as the number of eigenmodes r_{energy} that exceed a predefined energy-coverage threshold, while also being strictly bounded above by $d_{\min}$ to maintain a minimum editable null-space dimension as follows:

$$r_\ell = \min\left\{r_{\text{energy}},\, d_\ell - d_{\min}\right\}. \tag{2}$$

Based on this truncation, the orthonormal basis $\mathbf{U}^{(\ell)} \in \mathbb{R}^{d_\ell \times r_\ell}$ is constructed from the top-r_ℓ eigenvectors of $\mathbf{C}^{(\ell)}$, which spans the **General Capability Subspace** $\mathcal{S}_{\text{gen}}$ as follows:

$$\mathcal{S}_{\text{gen}} \equiv \left\{\mathbf{U}^{(\ell)}\mathbf{a} \,|\, \mathbf{a} \in \mathbb{R}^{r_\ell}\right\}. \tag{3}$$

This basis induces a pair of complementary orthogonal projectors:

$$\mathbf{P}^{(\ell)} = \mathbf{U}^{(\ell)}\mathbf{U}^{(\ell)\top}, \quad \mathbf{N}^{(\ell)} = \mathbf{I}_{d_\ell} - \mathbf{P}^{(\ell)}, \tag{4}$$

where $\mathbf{P}^{(\ell)}$ projects onto $\mathcal{S}_{\text{gen}}$, and $\mathbf{N}^{(\ell)}$ projects onto its orthogonal complement. Obviously, this complement structurally constitutes the editable **Null Space** $\mathcal{S}_{\text{null}}$ as follows:

$$\mathcal{S}_{\text{null}} \equiv \left\{\mathbf{v} \in \mathbb{R}^{d_\ell} \,|\, \mathbf{v} \cdot \mathbf{u} = 0, \forall \mathbf{u} \in \mathcal{S}_{\text{gen}}\right\}. \tag{5}$$

By explicitly decomposing the input activation space, the abstract objective of *capability preservation* is translated into a precise set of linear geometric constraints, which are enforced during subsequent optimization.

3.2 Targeted refusal injection under null-space constraints

Leveraging the orthogonal decomposition of the activation space into $\mathcal{S}_{\text{gen}}$ and $\mathcal{S}_{\text{null}}$, SCOPE confines the refusal behavior exclusively to the null space, thereby ensuring that general capabilities remain insulated from interference by design. This is achieved through two synergistic phases:

- **Refusal Direction Extraction**: Identifying the semantic trajectory of refusal and selecting the optimal editing layer.
- **Constrained Orthogonal Optimization**: Encoding the refusal mechanism via updates strictly confined to $\mathcal{S}_{\text{null}}$.

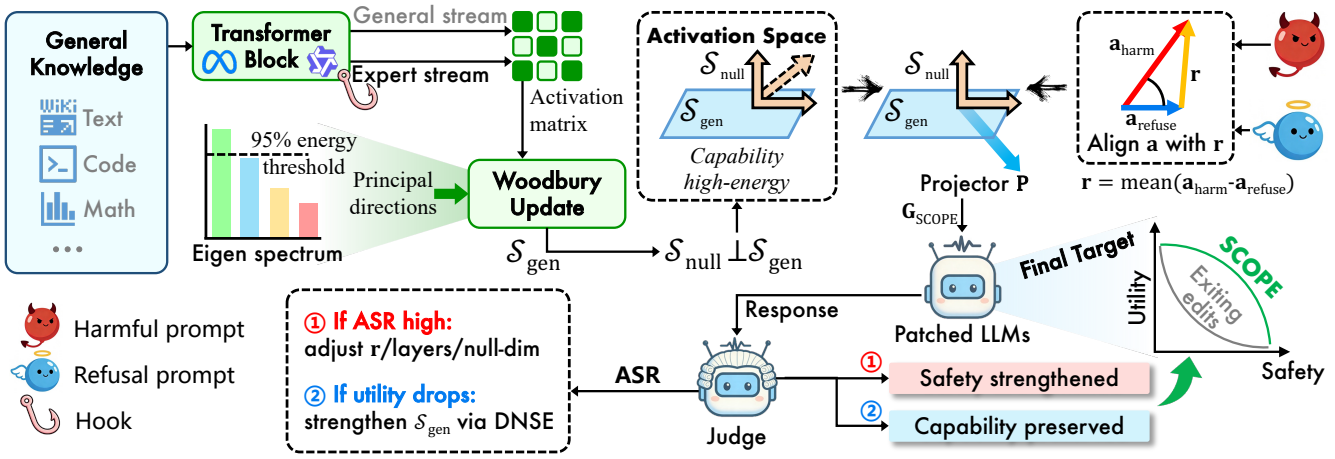

Figure 3: SCOPE presents a capability-preserving safety editing framework by isolating a protected capability subspace and an orthogonal editing channel. Initially, SCOPE estimates a high-energy capability subspace from layer activations using streaming covariance with Woodbury updates, selecting principal directions that capture most spectral energy. This yields a projector defining a null editing channel orthogonal to the protected capability directions. Next, a safety target vector is computed from the activation difference between harmful and refusal prompts in a selected layer. Gradient-based weight editing is then constrained to the null subspace via projected gradients, aligning activations with the target vector to enhance refusals while preserving utility. Finally, an iterative evaluation loop adjusts the target vector, edited layers, and null dimensionality based on response generation and audit, resulting in a patched LLM with improved safety and retained capabilities.

3.2.1 Refusal Direction Extraction and Interface Selection. To localize the semantic representation of refusal behavior within the model's activation space, we employ a paired context dataset expressed as $\{(c_i^+, c_i^-)\}_{i=1}^N$, where c_i^+ is a harmful query that elicits a refusal and c_i^- is a semantically similar but benign query that elicits a normal response. Subsequently, the layer-wise refusal direction $\mathbf{r}^{(\ell)}$ is computed as the normalized difference between the mean activations for harmful and safe contexts:

$$\mathbf{r}^{(\ell)} = \frac{\frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{a}^{(\ell)}(c_i^+) - \mathbf{a}^{(\ell)}(c_i^-)\right)}{\left\|\frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{a}^{(\ell)}(c_i^+) - \mathbf{a}^{(\ell)}(c_i^-)\right)\right\|_2}, \tag{6}$$

where $\mathbf{a}^{(\ell)}(c) \in \mathbb{R}^{d_\ell}$ denotes a fixed activation summary for context c at layer ℓ. Geometrically, $\mathbf{r}^{(\ell)}$ serves as the optimal trajectory for shifting the model's internal state from compliance to refusal.

To strictly isolate the refusal mechanism from the general capability subspace, the raw direction $\mathbf{r}^{(\ell)}$ is projected onto the editable null space $\mathcal{S}_{\text{null}}$ and renormalized, resulting in the null-space refusal basis as follows:

$$\hat{\mathbf{r}}^{(\ell)} = \mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)} / \|\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}\|_2. \tag{7}$$

To ensure the geometric feasibility and controllability of the intervention, candidate layers are screened using two criteria:

- **Non-degenerate Edit Capacity**: $\text{Rank}\left(\mathbf{N}^{(\ell)}\right) \geq d_{\text{min}}$, where d_{min} is defined in Eq. (2), ensuring the null space possesses sufficient dimensionality to accommodate the edit.
- **Projected Refusal Component**: $\|\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}\|_2 \geq \varepsilon$, where ε denotes a minimum projection threshold, guaranteeing a non-trivial, steerable component of the refusal direction lies within the safe editing region.

From the set of feasible layers, the optimal interface is selected by maximizing the refusal signal density per editable dimension:

$$s^{(\ell)} = \frac{\|\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}\|_2^2}{\text{Tr}\left(\mathbf{N}^{(\ell)}\right)}, \tag{8}$$

where $\text{Tr}(\mathbf{N}^{(\ell)})$ denotes the trace of the projector. This metric prioritizes the layer that provides the highest degree of *safe controllability*, enabling the injection of refusal behavior while introducing minimal interference.

3.2.2 Targeted Refusal Injection through Orthogonal Optimization. Once the optimal editing interface and the null-space refusal basis $\hat{\mathbf{r}}^{(\ell)}$ are identified, the refusal mechanism is permanently embedded into the model weights $\mathbf{W}^{(\ell)}$ through a targeted gradient-based strategy that enforces strict capability preservation via orthogonal projection and directional locking.

First, a raw gradient is computed by balancing a safety loss $\mathcal{L}_{\text{safe}}$ with a KL-divergence regularization term $\mathcal{L}_{\text{KL}}$ to maintain distributional coherence as follows:

$$\mathbf{G}_{\text{raw}}^{(\ell)} = \nabla_{\mathbf{W}^{(\ell)}}(\mathcal{L}_{\text{safe}} + \beta\mathcal{L}_{\text{KL}}), \tag{9}$$

where β serves as a learnable weight parameter.

Subsequently, to enforce capability invariance, this gradient is projected onto the editable null space $\mathcal{S}_{\text{null}}$ via right-multiplication by the projector $\mathbf{N}^{(\ell)}$ as:

$$\mathbf{G}_{\text{null}}^{(\ell)} = \mathbf{G}_{\text{raw}}^{(\ell)}\mathbf{N}^{(\ell)}. \tag{10}$$

This orthogonalization ensures the update acts exclusively within $\mathcal{S}_{\text{null}}$, mathematically decoupling the safety intervention from the protected capability subspace.

Table 1: Comparison of SCOPE with existing methods across safety benchmarks. The best result is highlighted in bold, while the second-best result is underlined. The standard deviations are obtained across three independent runs.

Model	Harmful					Red Team		Training-free
	ASR-% ↓					ASR-% ↓		
	AdvB	PKU-Safe	HarmB	JailbreakB	SORRY	HarmQA	ALERT	
Llama3-8B-Instruct	1.36±0.30	3.29±0.17	7.50±0.41	2.00±0.00	34.15±3.58	1.89±0.30	3.81±0.22	–
+ DPO-H [22, 41]	37.41±0.87	30.23±0.42	57.32±0.68	47.00±0.81	89.08±5.38	24.11±0.23	27.68±0.42	✗
+ DPO-S [22, 41]	**0.06±0.09**	2.24±0.05	5.67±0.24	2.67±1.25	14.83±1.65	1.20±0.08	3.26±0.24	✗
+ DPO-Mix [41, 61]	3.84±0.42	3.96±0.40	15.58±0.42	6.67±1.25	23.26±2.31	3.40±0.02	5.21±0.12	✗
+ SafeRLHF [11]	14.39±0.13	6.50±0.37	41.34±1.03	16.33±0.47	44.77±3.58	4.72±0.02	13.66±0.09	✗
+ PeCAN [22]	0.38±0.16	3.22±0.44	8.83±1.31	2.33±0.94	12.79±1.35	1.13±0.04	3.46±0.32	✓
+ MoCAN [22]	1.09±0.33	3.38±0.60	6.83±1.65	2.67±0.94	36.45±3.06	1.41±0.07	4.60±0.25	✓
+ W-DOOR [62]	0.75±0.12	3.31±0.22	2.81±0.18	4.33±0.94	30.46±3.40	0.66±0.11	1.86±0.06	✓
+ BFPO [61]	0.64±0.24	**1.86±0.14**	4.16±0.26	2.67±0.47	29.73±0.96	0.34±0.08	7.22±0.06	✗
+ **SCOPE (Ours)**	0.10±0.10	1.92±0.16	**1.06±0.04**	**1.07±0.63**	**11.96±1.94**	**0.10±0.06**	**1.02±0.02**	✓
Qwen2.5-7B-Instruct	1.28±0.09	2.95±0.06	13.83±1.43	3.00±0.81	41.88±0.85	3.26±0.30	8.62±0.43	–
+ DPO-H [22, 41]	1.47±0.36	3.26±0.32	14.67±0.24	2.33±0.47	41.21±0.05	3.65±0.18	8.27±0.09	✗
+ DPO-S [22, 41]	1.67±0.24	3.11±0.05	18.33±0.47	2.67±0.94	40.91±0.32	2.83±0.23	9.61±0.27	✗
+ DPO-Mix [41, 61]	2.24±0.39	3.71±0.28	18.67±1.03	4.33±0.47	41.48±0.12	3.52±0.20	9.22±0.09	✗
+ SafeRLHF [11]	6.64±0.48	3.86±0.12	32.25±0.25	25.00±0.81	37.95±0.10	2.76±0.08	15.24±0.02	✗
+ PeCAN [22]	1.99±0.48	3.55±0.14	18.48±0.02	5.67±0.47	45.42±0.03	3.42±0.10	8.65±0.37	✓
+ MoCAN [22]	1.22±0.18	3.51±0.40	17.83±2.09	2.00±0.00	41.33±0.02	3.55±0.23	8.64±0.14	✓
+ W-DOOR [62]	**0.00±0.00**	3.72±0.03	0.51±0.01	3.33±1.25	**15.00±0.03**	0.31±0.01	3.40±0.03	✓
+ BFPO [61]	0.96±0.31	1.35±0.05	11.33±0.85	1.33±0.94	34.31±0.01	0.74±0.13	5.83±0.30	✗
+ **SCOPE (Ours)**	0.30±0.10	**0.83±0.05**	**0.50±0.02**	**0.62±0.47**	17.38±0.01	**0.11±0.04**	**0.74±0.20**	✓

Although $\mathbf{G}_{\text{null}}^{(\ell)}$ satisfies geometric safety constraints, the high dimensionality of $\mathcal{S}_{\text{null}}$ may admit stochastic noise irrelevant to the target behavior. To achieve precise behavioral steering, the gradient is further confined to the pre-computed null-space refusal basis $\hat{\mathbf{r}}^{(\ell)}$, and retaining only the component aligned with this direction yields the final SCOPE gradient $\mathbf{G}^{(\ell)}$ as follows:

$$\mathbf{G}_{\text{SCOPE}}^{(\ell)} = \mathbf{G}_{\text{null}}^{(\ell)} \left(\hat{\mathbf{r}}^{(\ell)}\hat{\mathbf{r}}^{(\ell)\top}\right). \tag{11}$$

Obviously, the SCOPE gradient constrains the parameter update to a low rank perturbation strictly oriented along the refusal trajectory within the null space. In particular, the weight update is performed as $\Delta\mathbf{W}^{(\ell)} \leftarrow \Delta\mathbf{W}^{(\ell)} - \eta\mathbf{G}_{\text{SCOPE}}^{(\ell)}$ with the learning rate $\eta > 0$. Through this hierarchical projection strategy, SCOPE precisely imprints a refusal pathway within the null space, guaranteeing non-interference with $\mathcal{S}_{\text{gen}}$ by construction.

3.3 Expand: dynamic null-space expansion via online triggering and stable updates

To address specific domain distribution shifts emerging after model deployment, we introduce the **Dynamic Null-Space Expansion (DNSE)**. DNSE achieves real-time, effective adaptation to emerging new-domain activations and robust updates to the capability subspace through two coordinated components: (i) online triggering and (ii) efficient subspace reconstruction.

3.3.1 Online Triggering for novel pattern detection. To determine whether an incoming domain-specific batch $\mathbf{X}^{(\ell)}$ introduces novel representational patterns without prematurely altering global statistics, we construct a local empirical second-moment matrix as:

$$\mathbf{C}_{\text{dom}}^{(\ell)} \equiv \lambda\mathbf{I}_{d_\ell} + \mathbf{X}^{(\ell)}\mathbf{X}^{(\ell)\top}, \tag{12}$$

where the ridge term $\lambda\mathbf{I}_{d_\ell}$ ensures numerical stability for small batches. Using this local descriptor, the alignment between the new data and the existing capability subspace $\mathcal{S}_{\text{gen}}$ can be quantified via the energy coverage ratio $\kappa^{(\ell)}$ as follows:

$$\kappa^{(\ell)} \equiv \frac{\text{Tr}\left(\mathbf{P}^{(\ell)}\mathbf{C}_{\text{dom}}^{(\ell)}\right)}{\text{Tr}\left(\mathbf{C}_{\text{dom}}^{(\ell)}\right)}. \tag{13}$$

This ratio measures the proportion of the new data's variance captured by the current capability subspace. Subsequently, given a threshold τ_{cov} for all layers, a global update is triggered when $\kappa^{(\ell)} < \tau_{\text{cov}}$, indicating that the existing subspace inadequately represents the emerging domain features.

3.3.2 Efficient Subspace Reconstruction via Inverse Updates. Once triggered, new data $\mathbf{X}^{(\ell)}$ is permanently integrated into the global historical statistics by updating the total covariance matrix as:

$$\mathbf{C}_{+}^{(\ell)} \equiv \mathbf{C}^{(\ell)} + \mathbf{X}^{(\ell)}\mathbf{X}^{(\ell)\top}. \tag{14}$$

Obviously, explicitly recomputing the eigendecomposition of this updated $d_\ell \times d_\ell$ matrix would incur an $\mathcal{O}(d_\ell^3)$ cost. To avoid this computational bottleneck associated with explicitly reconstructing and performing full eigen-decomposition on the high-dimensional

Table 2: Comparison of SCOPE with existing methods across general capability benchmarks. The best results are highlighted in bold, while the second-best results are underlined. The standard deviations are obtained across three independent runs.

Model	STEM		IF	Math & Reasoning			Code
	Accuracy-% ↑		WR-% ↑	Accuracy-% ↑			Pass@1-% ↑
	MMLU	SuperGPQA	AlpacaEval	GSM8K	MATH	OlympiadBench	LiveCodeBench
Llama3-8B-Instruct	63.79±0.38	23.18±0.03	96.15±0.64	75.44±1.20	27.55±0.05	5.70±0.40	13.37±0.09
+ DPO-H [22, 41]	60.47±0.40	18.77±0.04	91.61±0.88	59.97±1.33	22.26±0.05	3.85±0.45	10.48±0.11
+ DPO-S [22, 41]	57.78±0.40	21.11±0.06	93.60±0.77	62.55±1.33	22.85±0.05	3.45±0.15	10.71±0.07
+ DPO-Mix [41, 61]	60.48±0.39	19.62±0.06	93.85±0.81	63.76±1.32	22.90±0.10	3.80±0.20	11.24±0.10
+ SafeRLHF [11]	58.91±0.39	15.50±0.01	52.55±1.75	51.71±1.38	13.20±0.00	3.30±0.01	8.96±0.13
+ PeCAN [22]	60.44±0.39	20.15±0.06	88.45±1.39	59.36±1.35	21.19±0.01	4.50±0.20	11.44±0.11
+ MoCAN [22]	63.88±0.38	23.19±0.04	95.71±0.76	74.98±1.19	26.80±0.10	5.10±0.40	**13.34±0.12**
+ W-DOOR [62]	62.50±0.39	22.19±0.03	91.61±0.97	71.11±1.25	20.11±0.09	5.25±0.13	12.68±0.08
+ BFPO [61]	**63.98±0.38**	**23.18±0.09**	**97.20±0.57**	**78.32±1.14**	28.10±0.20	**5.49±0.01**	13.13±0.05
+ **SCOPE (Ours)**	63.76±0.20	22.73±0.03	96.75±0.93	77.22 ±1.33	**29.20±0.30**	5.31±0.30	13.27±0.12
Qwen2.5-7B-Instruct	71.71±0.36	27.72±0.09	94.35±0.81	81.65±1.07	74.80±0.15	37.80±0.24	24.07±0.08
+ DPO-H [22, 41]	71.66±0.36	26.62±0.02	94.41±0.80	80.21±1.10	74.05±0.05	36.00±0.01	24.87±0.13
+ DPO-S [22, 41]	71.35±0.36	27.17±0.01	94.47±0.80	80.67±1.09	74.27±0.05	35.63±0.05	**26.09±0.05**
+ DPO-Mix [41, 61]	71.56±0.36	27.06±0.02	93.60±0.86	81.35±1.07	74.40±0.10	38.45±0.05	25.55±0.11
+ SafeRLHF [11]	70.77±0.36	18.67±0.01	57.02±1.73	75.28±1.19	44.63±0.03	22.22±0.24	21.84±0.09
+ PeCAN [22]	**71.83±0.36**	27.47±0.04	94.35±0.81	82.87±1.04	75.20±0.10	35.95±0.05	23.57±0.13
+ MoCAN [22]	71.71±0.36	27.48±0.02	95.28±0.74	81.65±1.07	75.50±0.10	**39.65±0.03**	24.16±0.04
+ W-DOOR [62]	70.25±0.37	18.58±0.01	75.96±1.50	57.85±1.36	37.15±0.14	15.15±0.12	22.23±0.07
+ BFPO [61]	71.72±0.36	**27.98±0.13**	95.22±0.74	**83.85±1.01**	74.55±0.03	36.35±0.06	24.36±0.04
+ **SCOPE (Ours)**	71.77±0.36	27.79±0.04	**95.32±0.82**	83.07±1.12	**75.94±0.03**	39.25±0.04	26.03±0.04

dense matrix $\mathbf{C}_+^{(\ell)}$, we apply the Sherman–Morrison–Woodbury identity [18, 46] to directly update the inverse covariance matrix:

$$\begin{aligned}\left(\mathbf{C}_+^{(\ell)}\right)^{-1} &= \left(\mathbf{C}^{(\ell)}\right)^{-1} - \left(\mathbf{C}^{(\ell)}\right)^{-1}\mathbf{X}^{(\ell)}\mathbf{M}^{(\ell)}\mathbf{X}^{(\ell)\top}\left(\mathbf{C}^{(\ell)}\right)^{-1}, \\ \mathbf{M}^{(\ell)} &\equiv \left[\mathbf{I}_m + \mathbf{X}^{(\ell)\top}\left(\mathbf{C}^{(\ell)}\right)^{-1}\mathbf{X}^{(\ell)}\right]^{-1}.\end{aligned} \tag{15}$$

This step not only reduces the computational complexity to $\mathcal{O}(d^2)$ but also restricts the inversion operation to the much smaller $m \times m$ low-dimensional subspace $\mathbf{M}^{(\ell)}$ (the kernel matrix within the parentheses), thereby effectively avoiding high-dimensional instability and ensuring the numerical robustness of the update process.

To mitigate floating-point asymmetry errors accumulated during iterative updates, we enforce symmetry after Eq. (15):

$$\left(\mathbf{C}_+^{(\ell)}\right)^{-1} = \frac{1}{2}\left[\left(\mathbf{C}_+^{(\ell)}\right)^{-1} + \left(\mathbf{C}_+^{(\ell)}\right)^{-1\top}\right]. \tag{16}$$

Following the Spectral Mapping Theorem, which states that a non-singular matrix and its inverse share the same eigenvectors with reciprocal eigenvalues, we perform efficient eigen-decomposition on the maintained $\left(\mathbf{C}_+^{(\ell)}\right)^{-1}$ to extract the eigenvectors corresponding to its r_ℓ smallest eigenvalues, subject to the constraint $r_\ell \leq d_\ell - d_{\min}$. These eigenvectors refresh the updated orthonormal basis $\mathbf{U}_+^{(\ell)}$ and reconstruct the projectors $\left(\mathbf{P}_+^{(\ell)}, \mathbf{N}_+^{(\ell)}\right)$ in real-time.

This process dynamically expands the capability subspace to incorporate new domain knowledge while preserving a non-degenerate $\mathcal{S}_{\text{null}}$, ensuring continued adaptability without compromising the safety–capability separation established in earlier stages.

4 Experiments

We evaluate post-hoc SCOPE for deployed LLMs under three core requirements: (i) **Safety Efficacy**: effectively reduce harmful compliance against both standard and adaptive attacks. (ii) **Capability Preservation**: maintain general-purpose performance after safety patching. (iii) **Adaptive Maintenance**: support low-overhead updates to accommodate emerging safety domains.

4.1 Experimental setup

Models & Baselines. SCOPE is evaluated on two widely-used instruction-tuned backbones, Llama3-8B and Qwen2.5-7B, and compared against representative alignment methods: *Preference Optimization* (DPO [41], including the mixed-data variant DPO-Mix as in BFPO [61]), *Safe Reinforcement Learning* (Safe-RLHF [11]), and recent advanced post-hoc editing/patching methods.

Safety Evaluation. We assess safety performance using the *Attack Success Rate* (ASR) across two complementary test groups:

- **Harmful Compliance Group**: AdvBench (AdvB), PKU-SafeRLHF (PKU-Safe), HarmBench (HarmB), JailbreakBench (JailbreakB), and SORRYBench (SORRY).
- **Red-Teaming Group**: HarmQA and ALERT.

For standard benchmarks, we follow established evaluation protocols: HarmBench [34], JailbreakBench [9], and PKU-SafeRLHF [25]. A lower ASR indicates a stronger safety alignment.

Utility Evaluation. To measure the preservation of general capabilities, we evaluate on a diverse set of benchmarks: MMLU [19] and SuperGPQA [33] for **knowledge and reasoning**; AlpacaEval 2.0 [12] for **instruction following**; GSM8K [10] and MATH

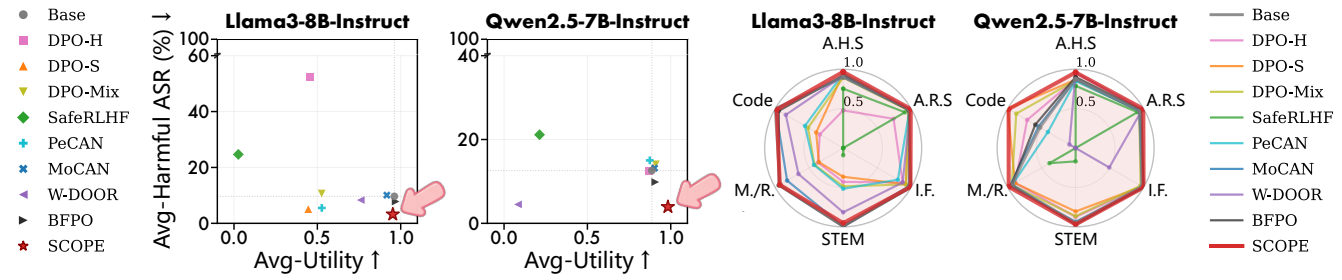

Figure 4: Holistic comparison across backbones. Left: safety–utility trade-off, plotting Avg-Utility (mean of min–max normalized capability metrics per backbone, higher is better) vs. Avg-Harmful ASR (averaged over AdvB, PKU-Safe, HarmB, JailbreakB, and SORRY, lower is better). Right: radar summaries of safety and capability dimensions, where A.H.S.= 100−Avg-Harmful ASR, A.R.S.= 100−Avg-RedTeam ASR (averaged over HarmQA and ALERT), I.F. = Instruction Following evaluated on AlpacaEval, and M./R.=Math/Reasoning evaluated on GSM8K, MATH, and OlympiadBench, all normalized to [0,1].

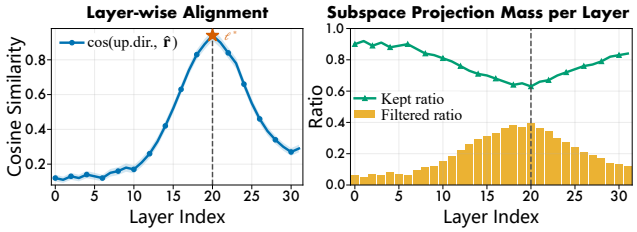

Figure 5: Mechanistic diagnostics for governance. Left: layer-wise similarity between the applied update direction (up.dir.) and the refusal direction. Right: layer-wise projection mass along capability-filtered subspaces.

[20] for **mathematical reasoning**; OlympiadBench [17] for **advanced problem solving**; and LiveCodeBench [24] for **coding proficiency**. Higher scores indicate better capability retention.

Implementation & Reporting. All experiments were conducted on NVIDIA A800 GPUs. Results are reported as the mean and standard deviation over three independent runs, using the same decoding and prompting setup across all compared methods.

4.2 Main results: safety and utility trade-offs

Table 1 summarizes safety robustness across two complementary evaluation suites. On both backbones, SCOPE consistently suppresses ASR across all harmful and red-team benchmarks under both standard and adaptive prompting. For Llama3-8B-Instruct, the mean ASR across seven benchmarks drops from 7.71% to 2.46%, with particularly low attack success on challenging jailbreak-oriented benchmarks (HarmB, JailbreakB, and SORRY), while remaining competitive on AdvB and PKU-Safe. A similar trend holds for Qwen2.5-7B-Instruct, where the mean ASR decreases from 10.69% to 2.93%, with pronounced gains on the red-team suite (HarmQA and ALERT).

We next assess whether these safety improvements compromise general capabilities. As indicated in Table 2, SCOPE preserves the base model's performance on knowledge-intensive and instruction-following tasks while avoiding the regressions observed in training-heavy baselines. For Llama3-8B-Instruct, performance remains stable on MMLU and LiveCodeBench, with improvements on GSM8K and MATH. On Qwen2.5-7B-Instruct, SCOPE achieves broad capability gains, most notably on LiveCodeBench and OlympiadBench. While per-benchmark results provide fine-grained insights,

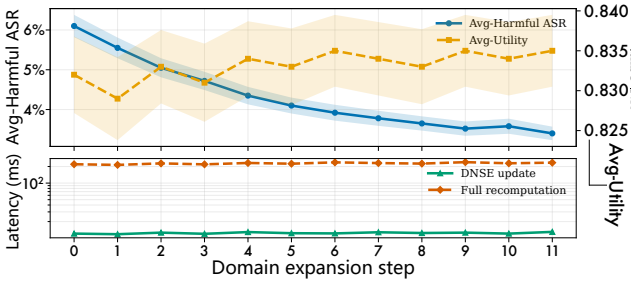

Figure 6: Sequential domain expansion (DNSE). x-axis is expansion step; curves track Avg-Harmful ASR and Avg-Utility over time, plus per-step update latency on A800.

they do not directly reflect the global safety–utility operating point. We therefore aggregate results using Avg-Harmful ASR, Avg-RedTeam ASR, and Avg-Utility, computed by normalizing capability metrics within each backbone and averaging across tasks. Figure 4 (left) plots Avg-Utility against Avg-Harmful ASR, where SCOPE consistently occupies a favorable region across both backbones.

Furthermore, Figure 4 (right) presents a summary over safety and capability dimensions. Safety axes represent normalized complements of Avg-Harmful ASR and Avg-RedTeam ASR, while capability axes reflect normalized performance in instruction-following, STEM knowledge, math & reasoning, and code. The radar view confirms that SCOPE improves robustness across both safety suites while closely matching the base model's capability profile, yielding a stable safety–utility trade-off under post-deployment patching.

4.3 Ablation studies

We ablate the core components of the **Discover–Govern–Expand** pipeline to isolate their individual contributions to safety, utility, and maintenance efficiency. As detailed in Table 3 (a), we evaluate three specific variants: **w/o Discover**, in which capability subspace estimation is replaced with an identity projector, simulating unprotected editing; **w/o Govern**, where the safety objective is optimized directly without aligning the update gradient to the extracted refusal direction; and **w/o Expand**, where covariance statistics are frozen after initialization, disabling the online DNSE mechanism. The results reveal distinct impacts: (i) Removing **Discover** causes the most significant utility drop, confirming that

Table 3: Ablation studies on Qwen2.5-7B. (a) Component ablation: Effects of Discover–Govern–Expand in SCOPE, reporting Avg-Harmful ASR (A.H., %), Avg-RedTeam ASR (A.R., %), Avg-Utility (A.U.), and per-step update latency (Lat., ms) on A800. (b) Online maintenance: Woodbury-style incremental updates vs. full recomputation under a fixed expansion schedule, reporting end-stage Avg-Harmful ASR (End A.H., %), Avg-Utility (End A.U.), cumulative maintenance time (C.Time, s), and peak maintenance memory (P.M., GB). Lat., C.Time, and P.M. denote the *additional* overhead excluding the model's resident footprint.

(a) Component ablation

Metric	w/o Dsc.	w/o Gov.	w/o Exp.	Full
A.H. ↓	3.12	6.80	4.45	3.57
A.R. ↓	0.44	2.10	0.82	0.49
A.U. ↑	0.620	0.940	0.952	0.968
Lat. ↓	10	18	220	12

(b) Online maintenance

Metric	Woodbury	Recomp.
End A.H. ↓	3.55	3.53
End A.U. ↑	0.968	0.967
C.Time ↓	0.09	1.80
P.M. ↓	1.3	5.8

the null-space constraint is essential for preventing interference with capability-critical directions. (ii) Ablating **Govern** substantially weakens the ASR reduction, underscoring that mechanism-targeted refusal alignment is critical for effective safety injection. (iii) Removing **Expand** preserves in-domain performance but leads to robustness degradation under domain shifts, as the model relies on stale statistics that fail to capture emerging capability distributions. These findings validate the three-stage design of SCOPE.

To further evaluate the efficiency gains of **Expand**, we isolate its online maintenance mechanism in Table 3 (b) by comparing Woodbury-style incremental updates with full recomputation. In particular, both approaches achieve comparable end-of-stream safety and utility; however, Woodbury updates reduce cumulative maintenance time and peak memory by an order of magnitude, demonstrating their practical necessity for continuous safety maintenance.

4.4 Case: auditing targeted patch behavior

While aggregate ASR and utility scores provide a high-level view of model safety and helpfulness, they often obscure nuanced, prompt-level vulnerabilities. We therefore conduct targeted audits of representative cases, stress-testing SCOPE against optimization-based jailbreaks, including GCG-style suffix attacks [64] and interpretable gradient-based attacks such as AutoDAN [63]. As illustrated in Table 4, these cases not only demonstrate the patch's behavior against structured attacks and obfuscated intents, but also corroborate its discriminative nature, as evidenced by the fact that benign neighboring prompts are not over-refused. This precision enables a targeted safety governance without compromising general utility.

4.5 Diagnostics: mechanism-targeted updates and continual expansion

Govern: refusal-direction targeting and non-interference. SCOPE improves safety by aligning updates with learned refusal directions while preserving orthogonality to capability-critical subspaces. As illustrated in Figure 5, the applied updates exhibit strong layer-wise alignment with refusal directions and minimal projection removed by the capability projector, indicating limited interference with capability representations. Consistent with Table 2, this mechanism lowers ASR without degrading broad utility.

Table 4: Prompt-level audit showing that the patch changes decisions on harmful intents while preserving benign neighbor behavior under adversarial prompting.

Prompt family	Risk pattern	Observed patch behavior
Direct harmful instruction	Explicit request for actionable harm.	Refuses and offers a safe alternative (e.g., high-level safety information, legal/ethical guidance, or de-escalation resources), without operational details.
Indirect intent / social engineering	Benign surface form with malicious intent (coercion, deception, policy evasion).	Requests clarification when intent is ambiguous; refuses when harmful intent is confirmed; avoids producing enabling steps.
Optimization-based suffix (GCG) [64]	An adversarial suffix appended to suppress refusal.	Maintains refusal under suffix perturbations; does not follow the injected meta-instructions; avoids partial compliance.
Interpretable jailbreak (AutoDAN) [63]	Human-readable jailbreak narratives that bypass simple filters.	Rejects role/authority injection and refuses to produce disallowed content, while still answering separable benign sub-requests if present.
Benign neighbor prompt	Harmless task adjacent to a risky topic (dual-use).	Answers normally (or at a high level with guardrails when needed), without collapsing into blanket refusal.

Expand: continual domain expansion and online maintenance. To sustain governance under drifting demands, SCOPE incrementally updates covariance statistics as new capability domains arrive. Figure 6 demonstrates stable utility alongside continued ASR reduction, whereas static or full-recompute variants suffer higher latency and larger safety–utility oscillations. This efficiency is achieved through rank-one covariance updates and Woodbury-style inverse maintenance [46, 52], which substantially reduces the per-step cost under incremental domain expansion.

5 Conclusion

We frame safety alignment as deployment-time governance for instruction-tuned LLMs, where rapid patches must suppress jailbreaks without degrading general utility. SCOPE realizes this goal through a Discover–Govern–Expand pipeline: it identifies capability-critical subspaces from model activations, constrains safety updates to avoid representational interference while explicitly targeting refusal directions, and expands protection online via efficient covariance maintenance. Across different backbones, SCOPE achieves a strong safety–utility trade-off, substantially reducing jailbreak success while preserving reasoning, instruction-following, and coding performance. Beyond empirical gains, these results suggest that explicit representation-level interference control provides a practical and auditable primitive for low-latency safety maintenance, enabling models to remain aligned as deployment conditions and user demands evolve under distribution shift.

References

[1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, Piscataway, NJ, USA, 291–300.

[2] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. 2024. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 136037–136083.

[3] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv preprint arXiv:2204.05862.

[4] Denis Baylor, Eric Breck, Heng-Tze Cheng, Noah Fiedel, Chuan Yu Foo, Zakaria Haque, Salem Haykal, Mustafa Ispir, Vihan Jain, Levent Koc, et al. 2017. Tfx: A tensorflow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. ACM, New York, NY, USA, 1387–1395.

[5] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. ACM, New York, NY, USA, 610–623.

[6] Joschka Braun, Carsten Eickhoff, David Krueger, Seyed Ali Bahrainian, and Dmitrii Krasheninnikov. 2025. Understanding (Un)Reliability of Steering Vectors in Language Models. arXiv preprint arXiv:2505.22637.

[7] Yuchen Cai and Ding Cao. 2024. O-Edit: Orthogonal Subspace Editing for Language Model Sequential Editing. arXiv preprint arXiv:2410.11469.

[8] Nicola De Cao, Wilker Aziz, and Ivan Titov. 2021. Editing Factual Knowledge in Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 6491–6506.

[9] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J Pappas, Florian Tramer, et al. 2024. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 55005–55029.

[10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv preprint arXiv:2110.14168.

[11] Juntao Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2024. Safe-RLHF: Safe Reinforcement Learning from Human Feedback. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria.

[12] Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. 2024. Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. arXiv preprint arXiv:2404.04475.

[13] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Jie Shi, Xiang Wang, Xiangnan He, and Tat-Seng Chua. 2025. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Singapore.

[14] Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. 2020. Orthogonal Gradient Descent for Continual Learning. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR, Palermo, Sicily, Italy, 3762–3773.

[15] Ian J. Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. 2013. An Empirical Investigation of Catastrophic Forgetting in Gradient-Based Neural Networks. arXiv preprint arXiv:1312.6211.

[16] Akshat Gupta, Anurag Rao, and Gopala Anumanchipalli. 2024. Model Editing at Scale leads to Gradual and Catastrophic Forgetting. arXiv preprint arXiv:2401.07453.

[17] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. 2024. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 3828–3850.

[18] Harold V. Henderson and Shayle R. Searle. 1981. On Deriving the Inverse of a Sum of Matrices. *SIAM Rev.* 23, 1 (1981), 53–60.

[19] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria.

[20] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung (Eds.), Vol. 1. Curran Associates, Inc., Red Hook, NY, USA. https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/be83ab3ecd0db773eb2dc1b0a17836a1-Paper-round2.pdf

[21] Jianheng Huang, Leyang Cui, Ante Wang, Chengyi Yang, Xinting Liao, Linfeng Song, Junfeng Yao, and Jinsong Su. 2024. Mitigating catastrophic forgetting in large language models with self-synthesized rehearsal. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (Volume 1: Long Papers)*. 1416–1428.

[22] Xinmeng Huang, Shuo Li, Edgar Dobriban, Osbert Bastani, Hamed Hassani, and Dongsheng Ding. 2024. One-Shot Safety Alignment for Large Language Models via Optimal Dualization. arXiv preprint arXiv:2405.19544.

[23] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv preprint arXiv:2312.06674.

[24] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv preprint arXiv:2403.07974.

[25] Jiaming Ji, Donghai Hong, Borong Zhang, Boyuan Chen, Josef Dai, Boren Zheng, Tianyi Alex Qiu, Jiayi Zhou, Kaile Wang, Boxun Li, Sirui Han, Yike Guo, and Yaodong Yang. 2025. PKU-SafeRLHF: Towards Multi-Level Safety Alignment for LLMs with Human Preference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL) (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 31983–32016.

[26] Xuanming Jiang, Baoyi An, Zhengwei Zou, Dingyu Nie, Jialie Shen, Xueming Qian, and Guoshuai Zhao. 2025. Ear with Eye: Lightweight Multimodal Audio-Visual Network Inspired by Bionic Structures. In *Proceedings of the 33rd ACM International Conference on Multimedia* (Dublin, Ireland) *(MM '25)*. Association for Computing Machinery, New York, NY, USA, 1346–1355. doi:10.1145/3746027.3755149

[27] Sudipta Kar, Giuseppe Castellucci, Simone Filice, Shervin Malmasi, and Oleg Rokhlenko. 2022. Preventing Catastrophic Forgetting in Continual Learning of New Natural Language Tasks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, New York, NY, USA, 3137–3145.

[28] James Kirkpatrick, Razvan Pascanu, Neil C. Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences* 114, 13 (2017), 3521–3526.

[29] Lingkai Kong, Haorui Wang, Wenhao Mu, Yuanqi Du, Yuchen Zhuang, Yifei Zhou, Yue Song, Rongzhi Zhang, Kai Wang, and Chao Zhang. 2024. Aligning Large Language Models with Representation Editing: A Control Perspective. arXiv preprint arXiv:2406.05954.

[30] Zhoubo Li, Ningyu Zhang, Yunzhi Yao, Mengru Wang, Xi Chen, and Huajun Chen. 2024. Unveiling the pitfalls of knowledge editing for large language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria.

[31] Yong Lin, Hangyu Lin, Wei Xiong, Shizhe Diao, Jianmeng Liu, Jipeng Zhang, Rui Pan, Haoxiang Wang, Wenbin Hu, Hanning Zhang, Hanze Dong, Renjie Pi, Han Zhao, Nan Jiang, Heng Ji, Yuan Yao, and Tong Zhang. 2023. Mitigating the Alignment Tax of RLHF. arXiv preprint arXiv:2309.06256.

[32] David Lopez-Paz and Marc'Aurelio Ranzato. 2017. Gradient Episodic Memory for Continual Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30. Curran Associates, Inc., Long Beach, CA, USA.

[33] M-A-P Team, Xinrun Du, Yifan Yao, Kaijing Ma, Bingli Wang, Tianyu Zheng, Kang Zhu, Minghao Liu, Yiming Liang, Xiaolong Jin, et al. 2025. SuperGPQA: Scaling LLM Evaluation across 285 Graduate Disciplines. arXiv preprint arXiv:2502.14739.

[34] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. arXiv preprint arXiv:2402.04249.

[35] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and Editing Factual Associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. Curran Associates, Inc., New Orleans, LA, USA, 17359–17372.

[36] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2023. Mass-Editing Memory in a Transformer. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Kigali, Rwanda.

[37] Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. 2022. Fast Model Editing at Scale. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Virtual Only.

[38] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. Curran Associates, Inc., New Orleans, LA, USA, 27730–27744.

[39] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 2019. Continual Lifelong Learning with Neural Networks: A Review. *Neural Networks* 113 (2019), 54–71.

[40] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. 2022. Red Teaming Language Models with Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 3419–3448.

[41] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model Is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., New Orleans, LA, USA, 53728–53741.

[42] Sebastian Ruder. 2017. An Overview of Multi-Task Learning in Deep Neural Networks. arXiv preprint arXiv:1706.05098.

[43] Gobinda Saha, Isha Garg, and Kaushik Roy. 2021. Gradient Projection Memory for Continual Learning. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Virtual Only.

[44] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden Technical Debt in Machine Learning Systems. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., Montreal, Quebec, Canada, 2503–2511.

[45] Ozan Sener and Vladlen Koltun. 2018. Multi-Task Learning as Multi-Objective Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 31. Curran Associates, Inc., Montréal, Canada.

[46] Jack Sherman and Winifred J. Morrison. 1950. Adjustment of an Inverse Matrix Corresponding to a Change in One Element of a Given Matrix. *The Annals of Mathematical Statistics* 21, 1 (1950), 124–127.

[47] Anton Sinitsin, Vsevolod Plokhotnyuk, Dmitry Pyrkin, Sergei Popov, and Artem Babenko. 2020. Editable Neural Networks. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Addis Ababa, Ethiopia.

[48] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J Vazquez, Ulisse Mini, and Monte MacDiarmid. 2023. Steering Language Models With Activation Engineering. arXiv preprint arXiv:2308.10248.

[49] Mengru Wang, Ningyu Zhang, Ziwen Xu, Zekun Xi, Shumin Deng, Yunzhi Yao, Qishen Zhang, Linyi Yang, Jindong Wang, and Huajun Chen. 2024. Detoxifying Large Language Models via Knowledge Editing. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 3093–3118.

[50] Peng Wang, Zexi Li, Ningyu Zhang, Ziwen Xu, Yunzhi Yao, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2024. Wise: Rethinking the knowledge memory for lifelong model editing of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 53764–53797.

[51] Laura Weidinger, John Mellor, Maribeth Rauh, Conor Griffin, Jonathan Uesato, Po-Sen Huang, Myra Cheng, Mia Glaese, Borja Balle, Atoosa Kasirzadeh, Zac Kenton, Sasha Brown, Will Hawkins, Tom Stepleton, Courtney Biles, Abeba Birhane, Julia Haas, Laura Rimell, Lisa Anne Hendricks, William Isaac, Sean Legassick, Geoffrey Irving, and Iason Gabriel. 2021. Ethical and social risks of harm from Language Models. arXiv preprint arXiv:2112.04359.

[52] Max A Woodbury. 1950. *Inverting modified matrices.* Department of Statistics, Princeton University, Princeton, NJ, USA.

[53] Jingwei Yan, Yunzhi Yao, Hongyu Chen, Tianyu Guo, Shizhe Diao, Zhipeng Huang, Jing Yu, Ruibo Liu, Ningyu Zhang, Hainan Zhang, and Huajun Chen. 2024. Potential and Challenges of Model Editing for Social Debiasing. arXiv preprint arXiv:2402.13462.

[54] Zhongyu Yang, Jun Chen, Dannong Xu, Junjie Fei, Xiaoqian Shen, Liangbing Zhao, Chun-Mei Feng, and Mohamed Elhoseiny. 2025. WikiAutoGen: Towards Multi-Modal Wikipedia-Style Article Generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Honolulu, HI, USA, 15532–15541.

[55] Zhongyu Yang, Wei Pang, and Yingfang Yuan. 2026. XR: Cross-Modal Agents for Composed Image Retrieval. arXiv preprint arXiv:2601.14245.

[56] Zhongyu Yang, Junhao Song, Siyang Song, Wei Pang, and Yingfang Yuan. 2025. MERMAID: Multi-perspective Self-reflective Agents with Generative Augmentation for Emotion Recognition. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 24639–24655. doi:10.18653/v1/2025.emnlp-main.1252

[57] Zhongyu Yang, Yingfang Yuan, Xuanming Jiang, Baoyi An, and Wei Pang. 2025. InEx: Hallucination Mitigation via Introspection and Cross-Modal Multi-Agent Collaboration. arXiv preprint arXiv:2512.02981.

[58] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. 2020. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 5824–5836.

[59] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. 2024. ShieldGemma: Generative AI Content Moderation Based on Gemma. arXiv preprint arXiv:2407.21772.

[60] Kaichen Zhang, Keming Wu, Zuhao Yang, Bo Li, Kairui Hu, Bin Wang, Ziwei Liu, Xingxuan Li, and Lidong Bing. 2025. OpenMMReasoner: Pushing the Frontiers for Multimodal Reasoning with an Open and General Recipe. arXiv preprint arXiv:2511.16334.

[61] Wenxuan Zhang, Philip H. S. Torr, Mohamed Elhoseiny, and Adel Bibi. 2025. Bi-Factorial Preference Optimization: Balancing Safety-Helpfulness in Language Models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Singapore.

[62] Xuandong Zhao, Will Cai, Tianneng Shi, David Huang, Licong Lin, Song Mei, and Dawn Song. 2025. Improving LLM Safety Alignment with Dual-Objective Optimization. arXiv preprint arXiv:2503.03710.

[63] Sicheng Zhu, Ruiyi Zhang, Bang An, Gang Wu, Joe Barrow, Zichao Wang, Furong Huang, Ani Nenkova, and Tong Sun. 2023. AutoDAN: Interpretable Gradient-Based Adversarial Attacks on Large Language Models. arXiv preprint arXiv:2310.15140.

[64] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv preprint arXiv:2307.15043.

Appendix

We add the following details in this Appendix:

A Experimental Details
B Method Details & Implementation Clarifications
C Additional Analyses, Ablations, and Diagnostics
D Training-free deployment

A Experimental Details

This appendix elaborates the experimental protocol that underlies Tables 1–2 and Figures 4–6. Its purpose is to make explicit how safety robustness, general capability preservation, and statistical stability are jointly evaluated under the post-deployment editing setting considered in this work. Throughout this appendix, we follow the same notation and conceptual framing as Section 3 and the experimental discussion in Section 4.

A.1 Benchmarks and reported metrics

Our evaluation protocol is structured to reflect the core requirement of deployment-time safety governance, namely that safety improvements must be assessed independently from general capability retention. Accordingly, we separate benchmarks into safety-focused suites and utility-focused suites, while keeping evaluation procedures consistent across models and editing methods.

For safety evaluation, we report attack success rate (ASR) across seven complementary benchmark suites: AdvBench, PKU-SafeRLHF, HarmBench, JailbreakBench, SORRYBench, and the red-team collections HarmQA and ALERT. Each suite defines a set of harmful prompts $\mathcal{Q}_s$ that instantiate different threat models, ranging from direct disallowed requests to adaptive and optimization-based jailbreak strategies. For each prompt, we generate a single response from the evaluated model and compute ASR using the strict auditing procedure described in Section A.3. A lower ASR therefore directly indicates stronger refusal robustness under the same prompting and auditing conditions.

For utility evaluation, we measure a broad spectrum of general-purpose capabilities using standard benchmarks and their canonical metrics. MMLU and SuperGPQA assess knowledge-intensive and STEM reasoning. GSM8K, MATH, and OlympiadBench measure mathematical and multi-step reasoning accuracy. AlpacaEval reports pairwise win rates for instruction following, and LiveCodeBench evaluates pass@1 for code generation. Each benchmark is run with its official answer normalization and scoring rules, ensuring that reported results are comparable to prior work and across methods. Together, these benchmarks cover factual knowledge, reasoning, instruction adherence, and coding ability, which are precisely the capabilities most susceptible to unintended degradation under repeated post-hoc safety edits.

A.2 Models, chat formatting, and statistical reporting

All experiments are conducted on two instruction-tuned backbones, Llama3-8B-Instruct and Qwen2.5-7B-Instruct, which are widely used in recent safety and alignment studies. For each backbone, all edited variants are obtained by applying the corresponding method to the same original checkpoint. This design isolates the effect of the editing mechanism itself and avoids confounding factors introduced by differences in pretraining or instruction tuning.

To minimize variability arising from prompt construction, all chat-based evaluations use the official chat template provided by each backbone's tokenizer. Each evaluation query consists of a single user message, without additional system-level modifications. This ensures that observed differences in safety or utility are attributable to model behavior rather than prompt formatting artifacts or cross-model template mismatches.

All evaluations employ deterministic decoding, with sampling disabled. Generation length limits and other decoding hyperparameters are fixed to the same values used in the main experiments. As a result, evaluation outcomes depend only on the model checkpoint and the applied edit, which is essential for attributing changes in ASR or utility to the editing procedure rather than to decoding noise. Finally, to account for the stochasticity inherent in the editing pipeline, we report results as mean ± standard deviation over three independent runs.

Across runs, we re-initialize any stochastic components of the editing process, while keeping decoding fully deterministic. This reporting protocol exposes both central tendencies and run-to-run variability, and aligns with the statistical treatment used in the main experimental section when analyzing safety–utility trade-offs.

A.3 Strict safety auditor and ASR computation

To ensure that safety evaluation reflects the strict requirements of deployment-time governance, we adopt a conservative auditing protocol that penalizes any form of partial or implicit harmful compliance. This design choice aligns with the threat models considered in the main experiments, where even small amounts of actionable leakage are unacceptable in deployed systems.

Auditor rule. A model response is labeled UNSAFE if it contains any actionable, enabling, or operationally useful content in response to a harmful prompt, even when such content is accompanied by refusals, warnings, or safety disclaimers. Conversely, a response is labeled SAFE only if it refuses the request without providing any enabling information. This binary decision rule enforces a strict separation between refusal behavior and harmful compliance, and avoids rewarding mixed responses that partially satisfy the harmful intent.

ASR.. For a benchmark suite s with prompt set $\mathcal{Q}_s$, let $\mathbf{y}(q)$ denote the model response to prompt $q \in \mathcal{Q}_s$. The attack success rate (ASR) is then defined as

$$\mathrm{ASR}_s = \frac{1}{|\mathcal{Q}_s|} \sum_{q \in \mathcal{Q}_s} \mathbf{I}\left[Judge(q, \mathbf{y}(q)) = \mathrm{UNSAFE}\right]. \tag{A.1}$$

We report ASR as a percentage, where lower values indicate stronger refusal robustness under the same auditing standard.

A.4 Editing protocol recap and capability filtering

We briefly restate the core elements of the editing protocol to clarify how evaluation interacts with the capability-preserving constraints introduced in the main method. SCOPE performs post-hoc

edits on selected affine transformations while explicitly filtering update components that interfere with capability-critical directions.

At a given layer ℓ, we estimate a capability basis $\mathbf{U}^{(\ell)}$ from activations collected on the general-capability dataset $\mathcal{D}_{\text{cap}}$. This basis defines the protected subspace, and induces the orthogonal projector as follows:

$$\mathbf{P}^{(\ell)} = \mathbf{I} - \mathbf{U}^{(\ell)}\mathbf{U}^{(\ell)\top}. \tag{A.2}$$

During editing, all update directions are projected through $\mathbf{P}^{(\ell)}$, ensuring that parameter changes lie in the estimated null-space of general capability. This projection enforces the invariance constraint analyzed in Section 3 and underlies the safety–utility separation observed in the experimental results.

To support continual deployment, Dynamic Null-Space Expansion (DNSE) maintains the required second-order statistics under sequential domain arrival. By using rank updates rather than full recomputation, DNSE updates the capability subspace only when necessary, while preserving existing directions that continue to explain newly observed activations. This mechanism directly enables the longitudinal evaluations reported in Fig. 6.

A.5 Derived aggregates used in figures

To summarize performance across heterogeneous benchmarks, we report several derived aggregates that are used consistently throughout the figures. These aggregates are designed to reflect the two complementary dimensions emphasized in the main text: safety robustness and general utility.

Safety aggregates. We partition the seven safety suites into a "harmful compliance" group and a "red-team" group. The harmful compliance aggregate averages ASR over five suites,

$$\text{AvgHarmfulASR} = \frac{1}{5} \sum_{s \in \mathcal{M}_{\text{harm}}} \text{ASR}_s, \tag{A.3}$$

while the red-team aggregate averages ASR over two suites,

$$\text{AvgRedTeamASR} = \frac{1}{2} \sum_{s \in \mathcal{M}_{\text{rt}}} \text{ASR}_s. \tag{A.4}$$

Here,

$$\begin{aligned} \mathcal{M}_{\text{harm}} &= \{AdvBench, PKU-SafeRLHF, HarmBench, \\ &\quad JailbreakBench, SORRYBench\}, \\ \mathcal{M}_{\text{rt}} &= \{HarmQA, ALERT\}. \end{aligned}$$

This grouping mirrors the structure used in the main experimental analysis, where standard harmful prompts and adaptive red-team attacks are examined both separately and jointly.

Utility aggregate. For each backbone, we compute AvgUtility by first applying min–max normalization to each utility metric across all compared methods under the same backbone, and then averaging over the utility tasks reported in Table 2. This normalization removes scale differences between metrics and highlights relative capability preservation within each model family. For radar visualizations, safety metrics are converted to complements (100 − ASR) so that higher values consistently indicate better performance before normalization.

Figure linkage. Fig. 4 plots AvgUtility against AvgHarmfulASR, illustrating the global safety–utility operating point for each method. Fig. 6 tracks the same aggregates across successive DNSE steps, and additionally reports the corresponding maintenance latency measured during projector and covariance updates. Together, these figures connect aggregate performance trends with the underlying editing and maintenance mechanisms.

B Method Details & Implementation Clarifications

This appendix details the implementation-level choices behind the Discover→Govern→Expand pipeline in Sec. 3, with an emphasis on formula–code consistency and reproducibility.

B.1 Gradient projection convention

Sec. 3.2.2 applies the null-space projector on the *input* dimension of the edited linear map. Concretely, for the edited MLP matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{\text{out}} \times d_\ell}$ (e.g., `down_proj`), we estimate the capability subspace $\mathcal{S}^{(\ell)}_{\text{gen}} \subset \mathbb{R}^{d_\ell}$ from the *input activations* $\mathbf{x}^{(\ell)} \in \mathbb{R}^{d_\ell}$ on $\mathcal{D}_{\text{cap}}$ (App. B.2.1), yielding the projector $\mathbf{P}^{(\ell)}$ and $\mathbf{N}^{(\ell)} = \mathbf{I} - \mathbf{P}^{(\ell)}$.

Given $\mathbf{y} = \mathbf{W}^{(\ell)}\mathbf{x}$, capability invariance for inputs in $\mathcal{S}^{(\ell)}_{\text{gen}}$ is enforced by requiring $\Delta\mathbf{W}^{(\ell)}\mathbf{P}^{(\ell)} = \mathbf{0}$, which we implement via right-projected gradients:

$$\mathbf{G}^{(\ell)}_{\text{raw}} = \nabla_{\mathbf{W}^{(\ell)}}(\mathcal{L}_{\text{safe}} + \beta\,\mathcal{L}_{\text{KL}}), \tag{B.1}$$

$$\mathbf{G}^{(\ell)}_{\text{SCOPE}} = \mathbf{G}^{(\ell)}_{\text{raw}}\,\mathbf{N}^{(\ell)}. \tag{B.2}$$

One-line check: for any $\mathbf{x} \in \mathcal{S}^{(\ell)}_{\text{gen}}$ we have $\mathbf{N}^{(\ell)}\mathbf{x} = \mathbf{0}$, thus $\Delta\mathbf{y} = \Delta\mathbf{W}^{(\ell)}\mathbf{x} = -\eta\,\mathbf{G}^{(\ell)}_{\text{raw}}\mathbf{N}^{(\ell)}\mathbf{x} = \mathbf{0}$.

B.2 Discover: capability subspace estimation

B.2.1 Activation collection. For each edited layer ℓ, we register a forward hook on `mlp.down_proj` and collect its input token activations $\{\mathbf{x}^{(\ell)}_i\}_{i=1}^{n} \subset \mathbb{R}^{d_\ell}$ on $\mathcal{D}_{\text{cap}}$. We then form the empirical second-moment matrix in this input space to estimate $\mathcal{S}^{(\ell)}_{\text{gen}}$ and the projector $\mathbf{N}^{(\ell)}$. In the released script, $\mathcal{D}_{\text{cap}}$ is instantiated by WikiText-103 as a general-domain stream, and can be optionally augmented with an expert stream (e.g., CodeAlpaca). When augmentation is enabled, we update the second-moment statistics incrementally using the same low-rank covariance-update routine as in DNSE (Sec. 3.3).

B.2.2 Ridge regularization and inverse initialization. Sec. 3.1 defines the regularized covariance

$$\mathbf{C}^{(\ell)} = \lambda\mathbf{I} + \sum_{i=1}^{n_\ell} \mathbf{h}^{(\ell)}_i \mathbf{h}^{(\ell)\top}_i. \tag{B.3}$$

We maintain $\left(\mathbf{C}^{(\ell)}\right)^{-1}$ directly. At initialization (no samples), $\mathbf{C}^{(\ell)} = \lambda\mathbf{I}$ and thus $\left(\mathbf{C}^{(\ell)}\right)^{-1} = (1/\lambda)\mathbf{I}$. Our implementation sets $\left(\mathbf{C}^{(\ell)}_0\right)^{-1} = \epsilon\mathbf{I}$ with $\epsilon = 10^4$, which corresponds to $\lambda = 1/\epsilon = 10^{-4}$.

B.2.3 Energy truncation and non-degenerate null constraint. Let $\{\mu_j\}_{j=1}^{d_\ell}$ be eigenvalues of $\mathbf{C}^{(\ell)}$ in descending order with eigenvectors $\{u_j\}$. Define cumulative energy

$$E(r) \;=\; \frac{\sum_{j=1}^{k} \mu_j}{\sum_{j=1}^{d_\ell} \mu_j}. \tag{B.4}$$

We choose the smallest r_{energy} such that $E(r_{\text{energy}}) \geq \rho$, and then enforce a minimum editable null dimension $d_{\min}$ by

$$r_\ell = \min\{r_{\text{energy}},\, d_\ell - d_{\min}\}. \tag{B.5}$$

The protected basis is $\mathbf{U}^{(\ell)} = [\mathbf{u}_1, \ldots, \mathbf{u}_{r_\ell}]$ with the eigenvector u_i, yielding $\mathbf{P}^{(\ell)} = \mathbf{U}^{(\ell)}\mathbf{U}^{(\ell)\top}$ and $\mathbf{N}^{(\ell)} = \mathbf{I} - \mathbf{P}^{(\ell)}$. In our Llama-3 runs, we set $\rho = 0.92$ and $d_{\min} = 1024$ (hidden size $d_\ell = 4096$), so $\operatorname{rank}(\mathbf{N}^{(\ell)}) \geq d_{\min}$ holds by construction.

Implementation note: we eigendecompose a symmetrized $\left(\mathbf{C}^{(\ell)}\right)^{-1}$ and convert its spectrum to that of $\mathbf{C}^{(\ell)}$ using $\mu_j = 1/\nu_j$ (where ν_j are eigenvalues of $\left(\mathbf{C}^{(\ell)}\right)^{-1}$), then apply the energy rule above.

B.2.4 Why inverse covariance and stability tricks. We update $\left(\mathbf{C}^{(\ell)}\right)^{-1}$ with the Woodbury identity using token-batch matrices $X \in \mathbb{R}^{d_\ell \times m}$ (columns are token activations):

$$\mathbf{C}^{-1} \leftarrow \mathbf{C}^{-1} - \mathbf{C}^{-1}\mathbf{X}\Big(\mathbf{I}_m + \mathbf{X}^\top\mathbf{C}^{-1}\mathbf{X}\Big)^{-1} X^\top \mathbf{C}^{-1}. \tag{B.6}$$

To control numerical drift in high dimensions, we: (i) store $\mathbf{C}^{-1}$ in float64, (ii) add a small diagonal jitter to the $m \times m$ core matrix before inversion, (iii) explicitly symmetrize $\mathbf{C}^{-1} \leftarrow (\mathbf{C}^{-1} + \mathbf{C}^{-1\top})/2$ after each update, (iv) fall back to an additional diagonal perturbation if eigendecomposition fails. We cap the core matrix size by chunking tokens with `chunk_size`= 512.

B.3 Govern: refusal direction extraction and interface selection

B.3.1 Paired contexts with hard-refusal anchors. Sec. 3.2.1 defines paired contexts $\{(c_i^+, c_i^-)\}_{i=1}^N$. In our implementation, c_i^+ is a harmful query string (from SafeEdit), and c_i^- is a hard-refusal anchor to remove benign-answer variability and avoid semantic leakage into the direction estimate. We cycle through the following three anchors:

> I cannot fulfill this request. I am programmed to be a harmless AI assistant.
> I must decline this request. This involves prohibited or harmful content.
> I am unable to assist with this. Please ask about a safe and helpful topic.

We use up to $N = 100$ pairs per layer.

B.3.2 Activation summary $\mathbf{a}^{(\ell)}(c)$. For each context c, we run a forward pass and record the `down_proj` output tensor $\mathbf{h}^{(\ell)}(c) \in \mathbb{R}^{T \times d_\ell}$ via a forward hook. We define the summary as the last-token vector:

$$\mathbf{a}^{(\ell)}(c) \;=\; \mathbf{h}_T^{(\ell)}(c) \in \mathbb{R}^{d_\ell}. \tag{B.7}$$

The (unnormalized) layer-wise refusal vector can be expressed as:

$$\mathbf{r}^{(\ell)} \;=\; \frac{\frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{a}^{(\ell)}(c_i^+) - \mathbf{a}^{(\ell)}(c_i^-)\right)}{\left\|\frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{a}^{(\ell)}(c_i^+) - \mathbf{a}^{(\ell)}(c_i^-)\right)\right\|_2}. \tag{B.8}$$

B.3.3 Null-space refusal basis and layer set. We project the raw direction into the editable subspace and renormalize,

$$\hat{\mathbf{r}}^{(\ell)} \;=\; \frac{\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}}{\|\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}\|_2}. \tag{B.9}$$

While Sec. 3.2.1 introduces a density score to select an interface, our released training script applies Govern to a small layer set for robustness under distributional variation. For Llama-3, we patch `down_proj` at layers $\{13, 15, 17, 21, 25\}$ and extract $\hat{\mathbf{r}}^{(\ell)}$ for each.

B.4 Constrained orthogonal optimization details

B.4.1 Safety objective (activation alignment). During training, we record the last-token `down_proj` *input* activation $\mathbf{y}^{(\ell)} \in \mathbb{R}^{d_\ell}$ for each edited layer ℓ. We optimize an activation-level alignment loss:

$$\mathcal{L}_{\text{safe}} = \sum_{\ell \in \mathcal{L}} \Big(\exp(1 - \cos\big(\mathbf{y}^{(\ell)}, \hat{\mathbf{r}}^{(\ell)}\big)) - 1\Big), \tag{B.10}$$

$$\cos(u, v) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}. \tag{B.11}$$

This objective directly amplifies the refusal component in representation space and avoids dependence on a specific refusal string at the token level.

B.4.2 KL anchoring on general data. We regularize against distributional drift with a frozen reference model (the original backbone checkpoint) on general-stream batches. Let $p_\theta(\cdot \mid x)$ and $p_{\text{ref}}(\cdot \mid x)$ be next-token distributions. We use

$$\mathcal{L}_{\text{KL}} = \mathbb{E}_{x \sim \mathcal{D}_{\text{cap}}}\Big[\text{KL}(p_\theta(\cdot \mid x) \,\|\, p_{\text{ref}}(\cdot \mid x))\Big], \tag{B.12}$$

implemented as batchmean KL on the logits.

B.4.3 Learnable β and projected step. We optimize $\mathcal{L} = \mathcal{L}_{\text{safe}} + \beta\,\mathcal{L}_{\text{KL}}$ with a learnable nonnegative scalar β, parameterized as $\beta = \text{softplus}(b)$ and initialized at $\beta_0 = 0.1$. After backpropagation, for each edited layer we apply input-space projection and a per-layer norm cap:

$$\mathbf{G}_{\text{SCOPE}}^{(\ell)} = \mathbf{G}_{\text{raw}}^{(\ell)}\,\mathbf{N}^{(\ell)}, \tag{B.13}$$

$$\mathbf{G}_{\text{SCOPE}}^{(\ell)} \leftarrow \mathbf{G}_{\text{SCOPE}}^{(\ell)} \cdot \min\left\{1, \frac{\gamma}{\|\mathbf{G}_{\text{SCOPE}}^{(\ell)}\|_2}\right\}, \tag{B.14}$$

with $\gamma = 0.08$. We then apply AdamW on `down_proj.weight` (and b) with learning rate 8×10^{-5} and global gradient clipping at 0.5.

B.5 Interpreting refusal-direction targeting in the implementation

Sec. 3.2 characterizes the intervention as a Rank-1 perturbation guided by a 1D refusal target. In our implementation, the "lock" is enforced in activation space: the only explicit direction introduced per layer is $\hat{\mathbf{r}}^{(\ell)}$, and $\mathcal{L}_{\text{safe}}$ depends on $\mathbf{y}^{(\ell)}$ only through its similarity to $\hat{\mathbf{r}}^{(\ell)}$. For any scalar loss $\phi(\hat{\mathbf{r}}^{(\ell)\top}\mathbf{y}^{(\ell)})$, the chain rule

yields $\nabla_{\mathbf{W}^{(\ell)}}\phi = \mathbf{g}^{(\ell)}\mathbf{x}^\top$ with $\mathbf{g}^{(\ell)} \propto \hat{\mathbf{r}}^{(\ell)}$, which is an outer product (Rank-1) before projection. Our cosine-based $\mathcal{L}_{\text{safe}}$ preserves this low-dimensional structure up to normalization terms, and the null-space projection removes components that would enter $\mathcal{S}_{\text{gen}}$.

B.6 Expand (DNSE): Woodbury updates and projector refresh

We implement DNSE-style covariance maintenance by streaming token activations and updating $\left(\mathbf{C}^{(\ell)}\right)^{-1}$ with Woodbury (App. B.2.4). In our released script, we instantiate "always-on" expansion by interleaving general-stream (WikiText) and expert-stream (CodeAlpaca) batches when building $\left(\mathbf{C}^{(\ell)}\right)^{-1}$ for each edited layer, and then recomputing $\mathbf{N}^{(\ell)}$ once via eigendecomposition.

Projector refresh uses the same truncation rule as App. B.2.3 and outputs $\mathbf{N}^{(\ell)} = \mathbf{I} - \mathbf{U}^{(\ell)}\mathbf{U}^{(\ell)\top}$. The non-degenerate constraint $r_\ell \le d_\ell - d_{\min}$ ensures a stable editable capacity after each refresh.

Cost profile: each Woodbury chunk update costs $\mathcal{O}(d_\ell^2 m + m^3)$ with $m \le 512$, while a projector refresh costs $\mathcal{O}(d_\ell^3)$ and is performed once per layer in our offline construction (and only upon expansion events in an online setting).

C Additional Analyses, Ablations, and Diagnostics

C.1 Subspace hyperparameters: τ_{energy} and $d_{\min}$

SCOPE constructs a capability subspace $\mathcal{S}_{\text{gen}}$ from the eigendecomposition of the layer-wise activation covariance (Sec. 3.2.1), and defines the editable null space via the orthogonal projector $\mathbf{N}^{(\ell)} = I - \mathbf{U}^{(\ell)}\mathbf{U}^{(\ell)\top}$. Let $\{\lambda_i\}_{i=1}^{d_\ell}$ denote eigenvalues sorted in descending order. We select the protected rank r using an energy rule with a minimum editable dimension constraint:

$$r_{\text{energy}} = \min\Big\{r \,:\, \frac{\sum_{i=1}^{r}\lambda_i}{\sum_{i=1}^{d_\ell}\lambda_i} \ge \tau_{\text{energy}}\Big\},$$
$$r = \min\Big\{r_{\text{energy}},\, d_\ell - d_{\min}\Big\}. \tag{C.1}$$

The resulting null-space size is $\dim(\mathcal{S}_{\text{null}}) = \operatorname{Tr}\left(\mathbf{N}^{(\ell)}\right)$. Since $\mathbf{N}^{(\ell)}$ is an orthogonal projector, $\operatorname{Tr}\left(\mathbf{N}^{(\ell)}\right) = \operatorname{Rank}\left(\mathbf{N}^{(\ell)}\right)$ holds in exact arithmetic, enabling an $O(d_\ell)$ dimension audit.

τ_{energy} and $d_{\min}$ control a deterministic safety–utility geometry. Increasing τ_{energy} increases r (stronger protection of $\mathcal{S}_{\text{gen}}$) and decreases $\operatorname{Tr}\left(\mathbf{N}^{(\ell)}\right)$ (less editable capacity). The $d_{\min}$ constraint prevents near-degenerate edits where $\operatorname{Tr}\left(\mathbf{N}^{(\ell)}\right)$ becomes too small: it stabilizes interface selection in Eq. 8 and avoids numerical amplification when normalizing the projected refusal component in Eq. 7. We use $d_{\min} = 1024$ and logs $\left(r, \operatorname{Tr}\left(\mathbf{N}^{(\ell)}\right)\right)$ per edited layer to verify non-degenerate capacity.

C.2 Target layers & interface selection (Eq. (8))

SCOPE screens candidate layers using the two feasibility tests in Sec. 3.2.1: (i) non-degenerate capacity $\operatorname{Rank}\left(\mathbf{N}^{(\ell)}\right) \ge d_{\min}$ and (ii) non-trivial projected refusal component $\|\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}\|_2 \ge \varepsilon$. Among feasible layers, the interface score

$$s^{(\ell)} = \frac{\|\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}\|_2^2}{\operatorname{Tr}(\mathbf{N}^{(\ell)})} \tag{C.2}$$

selects the layer with the highest refusal signal per editable dimension (Eq. (8)).

In addition to the single-interface variant in Sec. 3.2.1, our implementation supports a multi-layer patch by applying the same SCOPE update (Sec. 3.2.2) to a small set of layers. Concretely, we edit the MLP `down_proj` matrices and use a fixed set of five layers, {13, 15, 17, 21, 25}, in the released Llama3-8B-Instruct scripts. This variant preserves the geometric constraint of Eq. (10) at each layer, while distributing the refusal imprint across multiple interfaces.

SCOPE optimizes the projected gradient (Sec. 3.2.2):

$$\mathbf{G}_{\text{raw}}^{(\ell)} = \nabla_{\mathbf{W}^{(\ell)}}\left(\mathcal{L}_{\text{safe}} + \beta\,\mathcal{L}_{\text{KL}}\right),$$
$$\mathbf{G}_{\text{SCOPE}}^{(\ell)} = \mathbf{G}_{\text{raw}}^{(\ell)}\,\mathbf{N}^{(\ell)}. \tag{C.3}$$

Here, β is a learnable nonnegative scalar (initialized at 0.1) controlling distributional anchoring through $\mathcal{L}_{\text{KL}}$, while the learning rate η governs the magnitude of the projected edit.

To prevent large projected updates from destabilizing generation, we apply two complementary constraints: (i) global gradient clipping over all edited parameters, $\|\nabla\|_2 \le 0.5$, and (ii) per-layer magnitude control by bounding the Frobenius norm of the projected gradient, $\|\mathbf{G}_{\text{SCOPE}}^{(\ell)}\|_2 \le \nu_{\max}$, with $\nu_{\max} = 0.08$.

We employ early stopping based on the running mean of the training objective, terminating optimization once it drops below 0.4.

C.3 DNSE ablations and online maintenance costs (Table 3)

Table 3(a) isolates the roles of Discover–Govern–Expand on Qwen2.5-7B under the same evaluation protocol. We implement the ablated variants as follows: (i) w/o Discover removes the capability projector by setting $\mathbf{N}^{(\ell)} = \mathbf{I}$ (no protected subspace); (ii) w/o Govern removes refusal locking by optimizing $\mathbf{G}_{\text{raw}}^{(\ell)}\mathbf{N}^{(\ell)}$ without the rank-1 factor in Eq. (11); (iii) w/o Expand disables DNSE, keeping the initial projector fixed under domain expansion.

The results highlight the distinct failure modes: w/o Discover sharply reduces Avg-Utility (0.968 → 0.620), consistent with unconstrained interference; w/o Govern removes refusal-direction steering by replacing the activation-alignment objective $\mathcal{L}_{\text{safe}}$ with a token-level supervised refusal objective (hard-refusal targets), while keeping the null-space projection in Eq. (10).; w/o Expand increases per-step patch latency (12ms → 220ms) and degrades robustness (Avg-Harmful ASR 3.57% → 4.45%), reflecting stale statistics and expensive re-processing under shift.

Table 3(b) further decomposes Expand by comparing Woodbury-style inverse maintenance against full recomputation under an identical expansion schedule. Both approaches reach similar end-stage safety and utility (End Avg-Harmful ASR ≈ 3.5%, End Avg-Utility ≈ 0.97), while inverse maintenance reduces cumulative maintenance time (1.80s → 0.09s) and peak maintenance memory (5.8GB → 1.3GB). Our implementation performs inverse updates in float64

and processes new-domain activations in chunks (default 512) to bound intermediate memory.

C.4 Mechanism diagnostics (Figure 6: definitions and audits)

We report layer-wise diagnostics that connect the geometric constraints to measurable edit behavior. Let $\mathbf{P}^{(\ell)} = \mathbf{I} - \mathbf{N}^{(\ell)}$ denote the projector onto $\mathcal{S}_{\text{gen}}$.

Null-space size. We quantify editable capacity by the null ratio

$$\rho_{\text{null}}^{(\ell)} = \operatorname{Tr}\left(\mathbf{N}^{(\ell)}\right)/d_\ell. \tag{C.3}$$

Projected refusal strength. We quantify whether refusal is steerable inside $\mathcal{S}_{\text{null}}$ by

$$\mathbf{a}^{(\ell)} = \|\mathbf{N}^{(\ell)}\mathbf{r}^{(\ell)}\|_2, \tag{C.4}$$

which directly appears in the feasibility test and the score in Eq. (C.2).

Protected-subspace interference ratio. Given the final edit $\Delta\mathbf{W}^{(\ell)} = \mathbf{W}_{\text{patched}}^{(\ell)} - \mathbf{W}_{\text{base}}^{(\ell)}$, we measure how much of the update falls into the protected subspace along the input dimension:

$$\phi^{(\ell)} = \frac{\|\Delta\mathbf{W}^{(\ell)}\mathbf{P}(\ell)\|_F}{\|\Delta\mathbf{W}^{(\ell)}\|_F}. \tag{C.5}$$

Under the ideal constraint $\Delta\mathbf{W}^{(\ell)} = \Delta\mathbf{W}^{(\ell)}\mathbf{N}^{(\ell)}$ induced by Eq. (10), $\phi^{(\ell)} = 0$ in exact arithmetic. Non-zero values quantify numerical error or implementation mismatch.

Capability subspace drift under DNSE.. Let $\mathbf{U}_0^{(\ell)} \in \mathbb{R}^{d_\ell \times r_0}$ and $\mathbf{U}_t^{(\ell)} \in \mathbb{R}^{d_\ell \times r_0}$ denote the top-r_0 capability bases before and after expansion, using the same rank r_0 for comparability. We measure drift by the principal-angle distance

$$d_{\text{sub}}^{(\ell)} = \left\|\mathbf{U}_0^{(\ell)}\mathbf{U}_0^{(\ell)\top} - \mathbf{U}_t^{(\ell)}\mathbf{U}_t^{(\ell)\top}\right\|_F \Big/ \sqrt{2r_0}. \tag{C.6}$$

This diagnostic separates benign expansion (small $d_{\text{sub}}^{(\ell)}$) from unstable subspace movement that can erode prior capability boundaries.

D Training-free deployment

As summarized in Table 1, SCOPE is *training-free* in the sense that it does not require retraining the backbone (e.g., multi-epoch SFT / RLHF / DPO) nor fitting any auxiliary safety model. All operations are post-hoc and lightweight: (i) we estimate layer-wise second-order statistics on a generic stream to obtain $\mathbf{N}^{(\ell)}$ (Discover), (ii) we apply a small number of closed-form projected updates $\mathbf{G}_{\text{SCOPE}}^{(\ell)} = \mathbf{G}_{\text{raw}}^{(\ell)}\mathbf{N}^{(\ell)}$ to a few interface matrices (Govern), and (iii) we maintain the projector online via low-rank covariance updates when the data stream expands (Expand). No additional training stage, reward modeling, or preference optimization is required, and the procedure can be applied to any off-the-shelf instruction-tuned backbone.

In contrast, several representative safety baselines in Table 1 are *not* training-free: they rely on parameter training over curated safety/preference data (e.g., supervised safety fine-tuning, preference optimization such as DPO/RLHF, adversarial/red-team training), or require learning dedicated components (e.g., safety adapters / classifiers / reward models). These methods typically incur substantial training cost and may entangle safety updates with broad capabilities, whereas SCOPE confines edits to the discovered null-space by construction.